SCHEDULAB

CPU Scheduling Studio

Project questions, concepts, formulas, and presentation answers

A plain-language guide to how the simulator works, what its recommendations mean, and how to explain its contribution.

THE PROJECT IN ONE MINUTE

SCHEDULAB is an offline teaching simulator. You enter a set of processes and a goal, then it profiles the workload, recommends a scheduling policy with readable rules, simulates FCFS, SJF, Priority, and Round Robin, and compares their results. Its main value is helping students see both **why a policy was suggested** and **whether that policy actually scored best for the chosen goal**.

> **Important distinction:** The rules are transparent starting heuristics. The scorecard is a separate, workload-specific comparison of the four simulated policies. They can disagree.

CONTENTS

- 1. Workload inputs and scheduling terms
- 2. Scheduling policies and performance measures
- 3. Objectives, options, and Round Robin quantum
- 4. Decision rules: conditions, recommendations, and rationale
- 5. What the rating out of 100 means
- 6. Rule recommendation versus best-scoring policy
- 7. What-if lab and verification
- 8. Real-life use, target users, and project novelty

01 Workload inputs and scheduling terms

A process is a program that is ready to use or waiting for CPU time. A workload is the set of processes being scheduled.

Input / term	Meaning in SCHEDULAB
Process ID	A label used to identify a process in tables and the timeline.
Arrival time (AT)	The time when the process enters the ready system.
CPU burst (BT)	How much CPU time the process needs to complete. The simulator assumes this is known.
Priority	Importance assigned by the user. A smaller number means higher priority; for example, priority 1 runs ahead of priority 4 in Priority scheduling.
Process class	CPU-bound or interactive. This is a user-declared signal used by the heuristic; it is not inferred from a real operating system.

CPU-bound versus interactive

CPU-bound: Usually needs long stretches of computation, such as a video render or a large compile. **Interactive:** Often waits for a person or device, then benefits from getting CPU time soon, such as a text editor responding to a keystroke. Real programs can exhibit both kinds of behavior; the app simplifies them into one declared class.

Priority spread

Priority spread = largest priority number - smallest priority number. If priorities are 1, 2, 4, and 8, the spread is 7. It measures how wide the numeric range is, not how many jobs have each priority. R2 uses the spread only when Respect priorities is enabled.

> Example: priorities 2, 3, 4 give a spread of 2. Priorities 1, 2, 4 give a spread of 3, which reaches the current R2 threshold.

Four policies in simple terms

Policy	What it does	Typical trade-off
FCFS	Runs arrived processes in arrival order.	Simple baseline; a long job at the front can make short jobs wait (convoy effect).
SJF	Runs the arrived process with the shortest known burst; non-preemptive here.	Often reduces average waiting when burst lengths vary; long jobs can wait a long time.
Priority	Runs the arrived process with the highest priority (lowest number); non-preemptive here.	Honors declared importance; low-priority jobs can starve without aging.
Round Robin (RR)	Gives each ready process up to quantum q, then rotates unfinished work to the queue end.	Frequent turns can improve responsiveness; small q creates more context switches.

02 Performance measures

These measures describe different parts of a process journey. Lower is generally better except throughput and CPU utilization.

Metric	Plain-language meaning	Formula
CT - completion time	When the process finishes.	End of its final CPU segment
TAT - turnaround time	Total elapsed time from arrival until finish.	CT - AT
WT - waiting time	Time arrived but not using the CPU.	CT - AT - BT
RT - response time	Delay until the process first gets CPU time.	First start - AT
Throughput (X)	Number of completed processes per time unit.	process count / elapsed time
Context switches (CS)	Count of handoffs from one process to a different process. Idle gaps reset the previous process.	Counted handoffs
CPU utilization	Share of the elapsed schedule during which CPU work is happening.	busy time / elapsed time x 100%

Waiting time is not response time

A process may get a quick first turn, so its response time is low, then wait between later turns. Response time measures only the first start. Waiting time totals all time not running. Turnaround includes both CPU work and waiting.

Starvation and aging

Starvation means a process waits unusually long compared with the others. The project flags high waiting using a configured ratio to mean waiting time. With aging enabled, a waiting process gradually receives a better effective priority (smaller number) at five-time-unit intervals, reducing the chance it waits forever. Aging changes Priority scheduling; it does not make every policy preemptive.

Assumptions that affect the results

- One CPU is simulated.
- Burst times are known in advance.
- Context-switch counts are reported, but switch duration is zero.
- Process class is supplied by the user.
- This is a deterministic teaching simulator, not a live kernel scheduler.

03 Objectives, switches, and quantum

The selected objective changes how the scorecard values each metric. Objective cards are goals, not direct selections of a scheduler.

Objective	Weights: WT / RT / X / CS	In plain words
Minimize waiting	0.60 / 0.10 / 0.20 / 0.10	Waiting time has the largest influence.
Fast response	0.10 / 0.60 / 0.10 / 0.20	First CPU response matters most.
Balanced	0.25 / 0.25 / 0.25 / 0.25	All four measures count equally.
High throughput	0.10 / 0.10 / 0.60 / 0.20	Completed work per time matters most.

Priority controls

- **Respect priorities:** Allows a sufficiently wide priority spread to trigger R2 in the rule engine. The Priority policy simulation still uses the entered priorities.

- **Priority aging:** Gradually improves the effective priority of a waiting process to reduce starvation.

Round Robin quantum

The quantum q is the maximum CPU time given in one RR turn. If the process still needs CPU after q time units, it goes to the back of the ready queue. Adaptive q is calculated once from the mean burst:

Adaptive quantum: q = max(1, round(0.5 x mean burst))
Manual quantum: A user-entered positive whole number replaces the adaptive value.

A smaller q rotates sooner and can improve turn-taking, but increases switch count. A larger q reduces rotations but may make other jobs wait longer. Because switch time is zero in this model, the switch penalty is represented in the scoreboard, not in elapsed schedule time.

Useful notation

Symbol	Definition
rhoI	Interactive share = interactive process count / total process count.
CV	Burst coefficient of variation = burst standard deviation / mean burst. Dimensionless, so it does not depend on whether time is measured in ms or seconds.
rhoS	Short-job share = fraction of burst lengths below the mean burst.
Delta-pi	Priority spread = max priority - min priority.
Delta-alpha	Arrival spread = latest arrival - earliest arrival. Reported as context, not used by R1-R3.

04 Decision rules: where they come from and what they suggest

The rules come from the project papers as a transparent proposed design. They are not standard rules followed by every operating system.

Order	Condition	Recommendation	Why this signal can help
R1	Fast Response objective and rhoI >= 0.25; OR rhoI >= 2/3 for any objective.	Round Robin	Interactive work benefits from early and repeated CPU access.
R2	Respect priorities is on AND Delta-pi >= 3.	Priority	A wide priority range may mean the declared priorities should influence selection.
R3	CV >= 0.3 AND rhoS >= 0.5.	SJF	Variable burst lengths plus many short jobs can favor shortest-first scheduling.
R4	No earlier rule matches.	FCFS	A predictable arrival-order baseline when no specialist signal fires.

First match wins

The engine checks R1, then R2, then R3, then R4, and stops at the first match. This makes overlapping decisions deterministic. It also means an earlier rule can override a later signal. For example, R1 can choose RR even when R2 or R3 would also match.

Why these exact numbers?

The papers describe the thresholds as initial design values to calibrate against the scorecard, not as experimentally proven universal cutoffs. The 2/3 interactive threshold is tied to a 67% interactive example in the project specification. The other cutoffs are practical starting points: 0.25 means one-quarter, 3 means a range of three priority levels, 0.3 means moderate relative burst variation, and 0.5 means at least half the jobs are shorter than average.

How to describe them accurately: They are explainable project heuristics. Their directions have intuitive reasons, but their thresholds and precedence order are design choices that should be tested and calibrated.

Example from the project discussion

If rhoI = 0.75, R1 matches for any objective because 0.75 >= 2/3, so the rule pick is RR. If the selected goal is Minimize waiting and SJF scores higher after simulation, then SJF is the best-scoring choice while RR remains the heuristic pick. Both answers can be correct for their different questions.

05 How the rating out of 100 is calculated

The score is relative to the four policies simulated on the current workload. It is not a probability or a universal quality percentage.

Overall rating = 100 x (wWT x NWT + wRT x NRT + wX x NX + wCS x NCS)

N is a normalized metric score between 0 and 1. For lower-is-better metrics (WT, RT, CS):

N = (worst value - policy value) / (worst value - best value)

For higher-is-better throughput X:

NX = (policy throughput - lowest throughput) / (highest throughput - lowest throughput)

If all four policies tie on a metric, each receives N = 1 for that metric. The objective weights determine how much each normalized metric contributes.

Worked formula: Minimize waiting

For that objective, the weights are WT 0.60, RT 0.10, throughput 0.20, and context switches 0.10. A policy with normalized values NWT=0.8, NRT=0.6, NX=0.5, NCS=0.9 would score:

100 x (0.60 x 0.8 + 0.10 x 0.6 + 0.20 x 0.5 + 0.10 x 0.9) = 72 points

Rule pick versus best-scoring policy

Display	What it means
Rule engine pick	The policy selected by R1-R4 from workload profile signals.
Rule score	That rule-selected policy score under the chosen objective.
Best objective fit	The highest score after simulating all four policies.
Score gap / regret	Best score minus rule pick score, in score points, not seconds.

So if Priority has a rule score of 81.1 and SJF has the best score of 92.4, the gap is 11.3 points. For the selected objective, SJF is the best-scoring option in this model. The rule engine offers a separate, explainable heuristic suggestion.

Examples from the interface

Workload signal	Rule pick	Best fit for Minimize waiting	What the gap says
rhol = 0.75	RR	SJF: 94.3 vs RR: 30.0	64.3 points; RR is the heuristic choice, SJF scores best for waiting.
Delta-pi = 3; Respect priorities on	Priority	SJF: 92.4 vs Priority: 81.1	11.3 points; R2 selects Priority, while SJF scores best for waiting.

06 What-if lab and verification

The What-if Lab tests whether recommendations remain aligned as the workload changes.

Interactive share controls the proportion of generated processes marked interactive. Burst variability controls relative spread of burst lengths (CV). Priority spread controls the max-minus-min priority range. **Run 30 variations** generates repeatable scenarios while preserving process count and exploring arrivals, bursts, process class, and priorities. A tile can be selected to load that scenario into the editor.

Apply to workload changes the current workload using slider settings and runs the analysis. The 30-variation run is an experiment; by itself, it does not replace the main workload.

What the summary figures mean

Measure	Meaning
Best-policy counts	How often each policy received the highest objective score among the four.
Policy match / rule-best agreement	Whether the rule recommendation equals the best-scoring policy. In the current implementation these two labels calculate the same comparison, so they are redundant.
Average regret	Average best-score minus rule-score across all 30 scenarios. Large regret means the heuristic often differs substantially from the score winner.
Tile colors	Top grid indicates best-scoring policy; lower grid represents the rule recommendation. Hover shows both policies and the score gap.

Verification panel

The 10/10 badge is the worked example plus T1-T9 passing deterministic reference checks. They compare expected schedules, metrics, rule choices, and the aging effect. The green Success Criteria list describes project goals; it is not a separate exhaustive audit of every interface or browser. Passing these checks supports correctness for the covered cases but does not prove the simulator is bug-free for every possible input.

The six displayed goals are: correct schedules and metrics, explainable decisions, adaptive workload response, starvation handling, deterministic tie handling, and offline portability.

07 Real-life use, target users, and novelty

SCHEDULAB is designed for learning and demonstration, not for controlling a real machine.

Real-life style example

Imagine a lab workload containing a long video export, a code compile, a text editor, a music player, and a background backup. A student enters estimated arrivals, CPU bursts, priorities, and process classes, then selects Fast response. The app profiles the mix, shows the rule that fires, simulates FCFS/SJF/Priority/RR, and compares waiting, response, throughput, utilization, and switch counts. The student can then adjust the mix in What-if and inspect how the results change.

> This is a realistic educational simulation. It does not inspect actual OS processes, estimate live bursts, or install a scheduler in the operating system.

Target users

Primary users are operating-systems students and instructors. The studio helps students understand scheduling behavior, compare policies, and explain trade-offs. It can also support project demonstrations or introductory scheduler design discussions.

Project novelty

The contribution is not a newly invented base scheduling algorithm. The project combines classic policies with workload profiling, an explainable ordered-rule recommendation, objective-weighted scoring of all four simulations, regret comparison, starvation aging, and a what-if interface. The useful distinction is showing both why a heuristic chose a policy and whether that policy scored best for the user’s chosen goal.

A careful novelty claim

> "Our contribution is an explainable, objective-aware comparison and what-if simulator for CPU scheduling policies. It helps learners inspect the rule recommendation, compare it with the workload-specific score winner, and understand the trade-off."

How to present the rule-versus-score dilemma

"The heuristic is a transparent suggestion based on workload signals. The best objective fit is the highest scoring result after running all four policies against the selected goal. We show both because the rules can disagree with the simulated winner; the score gap makes that disagreement measurable."

Limitations to state honestly

- Known burst times are assumed; real systems must estimate future CPU bursts.
- Only one CPU is modeled, with zero time cost for context switches.
- Thresholds are heuristics and should be calibrated with broader workload experiments.
- The score winner is best among these four policies under this model and objective; it is not a universal proof of optimal scheduling.